Ders 1 Genişletilmiş Notu: Veri Sıkıştırmaya Giriş ve Temel Kavramlar

Ders 1 Genişletilmiş Notu: Veri Sıkıştırmaya Giriş ve Temel Kavramlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kaynaklar ve Dersin Konuları
- 2. Veri Sıkıştırma Nedir?
- 3. Sabit Uzunluklu ve Değişken Uzunluklu Kod
- 4. Artıklık (Redundancy)
- 5. Veri Sıkıştırmada Kullanılan Kavramlar
- 6. Algoritma Seçimi, Akış ve Blok Modu, Fiziksel ve Mantıksal Sıkıştırma
- 7. Sıkıştırma Oranı ve Performansı
- 8. Derlem (Corpus)
- 9. Görüntü Sıkıştırmada Performans Ölçümü
- Örnek Sorular

1. Kaynaklar ve Dersin Konuları

1.1 Kaynaklar

Slayttaki kaynak listesinde üç kitap vardır (ekrandan, 00:25:50):

- Khalid Sayood, *Introduction to Data Compression*
- J. Gibson, T. Berger, T. Lookabaugh, *Digital Compression for Multimedia*
- D. Salomon, *Data Compression*

Hoca derslerde ağırlıklı olarak Salomon'un *Data Compression* kitabını kullandığını söyledi. Her konuyu bu kitaptan anlatmıyor; bazı konuların başka kaynaklarda daha kolay anlatıldığını düşündüğü için o kaynakları tercih ediyor. Anlatılan her konuyla ilgili internette çok sayıda kaynak bulunabilir.

Hoca bu dersi önceki yıllarda tahtada anlattığını, ilk haftaların konularının daha çok sözlü anlatılan kavramlar olduğunu belirtti. Bu yüzden derste slaytın üzerine elle notlar ve şekiller çizerek ilerledi.

1.2 Dersin Konuları

Slayttaki konu listesi şöyledir (ekrandan, 00:40:00):

- Veri sıkıştırma nedir?
- Temel teknikler (sezgisel yöntemler, Run Length Coding, Move-To-Front)
- Hata bulma ve düzeltme
- Statik kodlar (prefix kodlar, Golomb kodu)
- İstatistiksel yöntemler (Shannon-Fano, Huffman ve türevleri, MNP ve türevleri, aritmetik kodlama, PPM)
- Sözlüksel yöntemler (LZ ve türevleri)

Hoca, Run Length Coding ve Move-To-Front'u önemli temel algoritmalar olarak andı. Hata bulma ve düzeltme bölümünde eşlik (parity) bitleri ve CRC işlenecek; hoca bu bölümü temel tekniklerin önüne almayı düşündüğünü söyledi ve slaytta bunu bir okla gösterdi.

Statik kodlar konusunda çok sayıda kodlama tekniği bulunduğunu, bunlardan en önemli ve en çok tercih edilenlerin prefix kodlar ile Golomb kodu olduğunu söyledi.

Hoca veri sıkıştırmada daha çok metin (text) üzerinde çalıştığı için metin kısmına daha fazla önem verdiğini belirtti. İstatistiksel yöntemler Shannon-Fano, Huffman ve türevleridir. "Türevler" ile kastedilen, Huffman kodlamasının statik, adaptif ve yarı adaptif (semi-adaptive) biçimleri ve Huffman'ın başlık (header) bilgisini kısaltmak için yapılan kodlamalardır. MNP, Microcom Network Protocol'dür; 5 ve 7 gibi farklı sürümleri vardır ve aralarındaki farklar işlenecek. Bunlara aritmetik kodlama ve PPM (Prediction by Partial Matching) eklenir. Bu yöntemlerin hepsinin ortak noktası, elimizdeki veri ister metin ister görüntü olsun, sembollerin (karakterlerin, piksellerin) olasılık bilgisini kullanarak çalışmalarıdır.

Sözlüksel yöntemlerde olasılıktan vazgeçilir; verideki desenler (pattern) bulunarak sıkıştırma yapılır. LZ adı iki araştırmacının, Lempel ve Ziv'in soyadlarından gelir. Bu adın sonuna eklerin gelmesiyle algoritmanın çok farklı türevleri oluşur.

> **Ek bilgi (derste anlatılmadı):** PPM'in açık adı kaynaklarda genellikle "Prediction by Partial Matching" olarak geçer. LZ ailesinin bilinen türevleri arasında LZ77, LZ78 ve LZW bulunur; sondaki harf ya da yıl, türevi ayırt eder.

Konular arasında ayrıca bir bilgi teorisi (information theory) bölümü olacaktır. Hocaya göre bu bölümde sıkıştırmanın "olmazsa olmaz" kavramı **entropi**dir. Entropi hem istatistiksel olarak hem de Markov modelleri üzerinden anlatılacak.

2. Veri Sıkıştırma Nedir?

2.1 Sıkıştırmanın Tanımı

Hoca sıkıştırmayı günlük bir benzetmeyle başlattı: dar bir alana çok fazla şey sığdırmak istediğimizde onları sıkıştırırız. Elimizde büyük bir veri ve dar bir alan vardır; veri üzerinde öyle işlemler yapmalıyız ki daha küçük bir alana sığsın.

Veri sıkıştırmanın iki temel kavramı **girdi akışı** (input stream) ve **çıktı akışı**dır (output stream). Sıkıştırma, girdi akışındaki veriyi alıp boyutunu küçültmek ve küçültülmüş veriyi çıktı akışına yazmaktır. Açma (decompress) işleminde çıktı akışı alınır ve girdi akışı yeniden oluşturulmaya çalışılır; bu, verinin aynısı ya da biraz eksik hâli olabilir. Ekrandaki şema şudur (ekrandan, 00:50:00):

```
IS --compress--> OS
OS --decomp----> IS
```

"Stream" akış demektir: veri sürekli bir noktadan gelir. Girdi akışına kaynak akış (source stream) da denir. Ham (raw), işlenmemiş, orijinal veri girdi akışı olarak geçer. Akış bir dosya ya da bir tampon (buffer) alanı olabilir; önemli olan içinde işlenmemiş verinin durduğu bir bölge olmasıdır. Bu veri farklı algoritmalarla sıkıştırılıp çıktı olarak alınır.

Hoca burada önemli bir şartı vurguladı: Bir veriyi sıkıştırdıktan sonra açtığımızda orijinal hâline (ya da kabul edilebilir hâline) ulaşamıyorsak durum kötüdür.

> **Hocanın vurgusu:** Sıkıştırılan veri açılamıyorsa sıkıştırma yapılmamış, veri kaybedilmiştir. Hoca bunu kilitlenip çıkılan ama sonra içine girilemeyen bir eve benzetti: kapalı kalan evin hiçbir anlamı yoktur. Her sıkıştırma algoritmasının mutlaka bir açma, yani ters işlemi olmalıdır.

2.2 Neden Sıkıştırırız?

Hocaya göre iki belirgin sebep vardır:

- **Fiziksel kazanç (alan):** Bilgisayara sürekli bir şeyler indirir ve kaydederiz; bir noktadan sonra fiziksel alan dolar, neyi nerede sakladığımızı bilmeden klasör silmeye başlarız. Sı-

kıştırmanın ilk amacı yerden kazanmaktır: 100 birimlik bir şeyi 30 birimlik yerde saklayıp alanı daha verimli kullanmak.
- **Hız:** Sıkıştırılan veri yalnız diskte saklanmaz, A noktasından B noktasına da gönderilir. Veri büyüdükçe yükleme ve indirme süresi uzar. Veri sıkıştırılarak gönderilir ya da sıkıştırılmış veri indirilirse zamandan da kazanılır.

3. Sabit Uzunluklu ve Değişken Uzunluklu Kod

Hoca slayttaki üç kavramı, sabit uzunluklu kod (fixed size code), artıklık (redundancy) ve değişken uzunluklu kod (variable size code), dersin geri kalanı için çok önemli olarak niteledi (ekrandan, 00:50:00).

3.1 Sabit Uzunluklu Kod

Sabit uzunluklu kod, her sembolü (karakteri, pikseli) hep aynı boyutta bir kodla kodlar. En bilinen örnek ASCII'dir: her karakter 8 bittir. "A" da 8 bit, "J" de 8 bit, nokta, ünlem, çift tırnak da 8 bittir. A'nın kodu her zaman 65'tir. Sabit uzunluklu kodların avantajı, kodlamanın ve kod çözmenin çok kolay olmasıdır.

Peki bu ideal bir kodlama mıdır? Türkçede en çok kullanılan karakter "a", en az kullanılan karakter "j"dir (İngilizcede en sık karakter belki "t"dir). Sabit uzunluklu kodda ikisi de 8 bitle kodlanır; oysa "a"nın çok daha sık geldiği bilinmektedir.

3.2 Değişken Uzunluklu Kod ve Sayısal Örnek

Değişken uzunluklu kodda sıkıştırmanın temel ilkesi uygulanır: **en sık geçen sembol daha az bitle, en az geçen sembol daha çok bitle kodlanır.**

Hocanın örneği (ekrandan, 00:53:40):

- **Verilen:** Bir metin dosyasında 30 tane "a" ve 2 tane "j" vardır.
- **Sabit uzunlukta (ASCII, 8 bit):**
  - "a": $30 \cdot 8 = 240$ bit
  - "j": $2 \cdot 8 = 16$ bit
  - Toplam: $240 + 16 = 256$ bit
- **Değişken uzunlukta** ("a" sık geçtiği için 3 bit, "j" seyrek geçtiği için 9 bit):
  - "a": $30 \cdot 3 = 90$ bit
  - "j": $2 \cdot 9 = 18$ bit
  - Toplam: $90 + 18 = 108$ bit
- **Sonuç:** 256 bit yerine 108 bit; değişken uzunluklu kod daha iyi sıkıştırır.

Hoca bu örnekteki 3 ve 9 bit değerlerini "kafadan attığını" belirtti. Gerçekte bu uzunluklar elle seçilmez. Huffman, aritmetik kodlama, LZW, PPM, Shannon-Fano gibi bir kodlama yöntemi seçilir ve o yöntemin verdiği bit uzunlukları kullanılır. Bazı kodlar da belirli bir algoritmik yapıya göre oluşturulmuştur. Değişken uzunluklu kodlarda kod uzunluğu, sembolün kullanım frekansıyla ters orantılıdır.

Burada "sembol" geniş anlamda kullanılır: bir karakter, karakter çiftleri ya da görüntüdeki bir piksel olabilir. Sembol, kodlanacak girdi birimidir.

4. Artıklık (Redundancy)

Redundancy'nin Türkçesi artıklıktır. Hoca, formülü entropi anlatılmadan verilemeyeceği için kavramı sezgisel olarak açıkladı.

Bir sembolü kodlayabilecek bir alt sınır vardır. Diyelim ki "a" 8 bitle kodlanıyor. Bir algoritma onu 5 bite, başka bir algoritma 3 bite indiriyor ve 3 bitten daha aza indiren bir algoritma yok (ekrandan, 00:59:10: $8 \to 5 \to 3$). 8'i 5'e indiren algoritma seçilirse ideal bir kodlama yapılmamış demektir; artıklık büyük kalır. 8'i 3'e indiren algoritma seçilirse artıklık küçük olur.

İkinci örnek boşluklarla ilgilidir. Bir metinde kelimeler arasında çok sayıda gereksiz boşluk olsun. Bir algoritma yalnız birkaç boşluğu siler; veri sıkışmıştır ama artıklık hâlâ büyüktür.

Kelimeler arasında yalnız bir boşluk kalana kadar bütün fazla boşlukları silen bir algoritma seçilirse atılacak fazlalık kalmaz ve artıklık küçülür.

Buradan şu sonuçlar çıkar:

- Artıklık ne kadar küçükse o veri için seçilen algoritma o kadar iyidir. Artıklık büyükse yanlış algoritma seçilmiştir; veride hâlâ sıkıştırılabilecek bir durum vardır.
- En ideal yöntem seçilmişse artıklık sıfıra yaklaşır; seçilmemişse sıfırdan büyür.
- Bir veriye tek bir algoritma uygulamak zorunlu değildir. Artıklık büyükse sıkıştırılmış verinin üzerinden başka bir algoritma geçirilebilir; artıklık biraz daha küçülür. Yine sıfıra yaklaşmamışsa bir algoritma daha uygulanabilir.
- Bunun bir durma noktası vardır: artıklık sıfıra yakın bir değere indiğinde artık başka bir yöntem uygulanamaz.

  Hocanın vurgusu: Art arda sıkıştırma uygulayıp elindeki veriyi bir bayta düşürmek diye bir şey yoktur. Bir bayta düşen veriden orijinale dönülemiyorsa sıkıştırma yapılmamış, veri kaybedilmiştir.

5. Veri Sıkıştırmada Kullanılan Kavramlar

Arada hoca o ana kadar anlatılanları özetledi: veri sıkıştırma girdi akışındaki veriyi sıkıştırıp çıktı akışına yerleştirmektir; amacı yerden ve zamandan kazanmaktır; değişken uzunluklu kodlarda kodlar algoritmalarla üretilir ve sembolün kullanım frekansıyla ters orantılıdır. Ardından sürekli kullanılacak terimlerin listesine geçti (ekrandan, 01:30:00). Slayttaki bazı terimler eğik çizgiyle ayrılmıştır, çünkü kaynaklar aynı kavrama farklı adlar verir.

5.1 Kompresör, Dekompresör, Codec

- **Compressor / Encoder / Kodlayıcı:** Girdi akışından veriyi alıp sıkıştıran ve çıktı akışına koyan algoritmadır.
- **Decompressor / Decoder / Kod Çözücü:** Kodlayıcının yaptığı işin tam tersini yapar; sıkıştırılmış veriyi açar. Her kompresörün bir ters işlemi olmalıdır.
- **Codec:** İkisinin birleşimidir. Hoca bunu kapalı bir kutu gibi düşünmeyi önerdi: veri verilir, sıkıştırılır, A noktasından B noktasına gönderilir ve B noktasında açılır. Kutunun içindeki bu bütün olaya codec denir.

5.2 Ham ve Kodlanmış Veri

- **Unencoded / Raw Data / Original Data:** Kodlanmamış, işlenmemiş ham veridir; bir metin, görüntü ya da ses dosyası olabilir.
- **Encoded / Compressed:** Kodlanmış, sıkıştırılmış veridir.

5.3 Adaptif Olmayan ve Adaptif Sıkıştırma

Non-adaptive compression (adaptif olmayan sıkıştırma) yöntemleri yalnız belli tipteki veriyi sıkıştırır; her tip veri bu yöntemlere verilemez. Örnek, faks cihazlarındaki sıkıştırma algoritmalarıdır. Bir anlaşma metni faks makinesine verildiğinde karşı tarafa metin olarak değil görüntü olarak gider: makine metnin fotoğrafını çeker. Metindeki her siyah ya da beyaz nokta bir piksel olarak işlem görür ve piksel üzerinde çalışan bir algoritma kullanılır. Bu algoritmaya doğrudan bir metin ya da ses verisi verilirse çalışmaz. İleride bir faks (facsimile) sıkıştırma algoritması gösterilecek.

Adaptive compression (adaptif sıkıştırma) her türlü veri üzerinde çalışır: metin, ses ya da görüntü fark etmez. Parametreler ve işlemler algoritma gereği değiştirilebilir. Hocanın örneği: sıkıştırmanın başında “a” sembolüne `111` kodu verilmiş olsun; metnin ortasında “a”nın kodu `10` olabilir, sonunda yine başka bir şey olabilir. Girdi akışı işlenirken değişiklikler olur ve algoritma buna uyum sağlar.

Slayttaki yazım hatası (“Adaptavie”) derste düzeltildi; doğrusu “Adaptive”dir.

5.4 Statik, Yarı Dinamik ve Dinamik Kodlama

Hoca adaptif sıkıştırmanın yanına bir parantez açarak üç kodlama türünü ayırdı (ekrandan, 01:40:00):

- **Statik:** Sıkıştırmaya başlamadan önce bütün sembollerin kodları belirlenir ve dokümanın sonuna kadar hep aynısı kullanılır. Kodlar önceden hazırlanmış durur. Örnek: ASCII tablosundan hazır kod alıp kullanmak.
- **Yarı dinamik (semi-adaptive):** Statiğin bir adım ötesidir ve iki aşamalıdır. Önce sembollerin frekansları, yani kaçar kez kullanıldıkları çıkarılır. İkinci aşamada sık gelen sembollere daha az, seyrek gelenlere daha çok bitle kod verilir.
- **Dinamik:** Aynı sembolün dokümanın başındaki, ortasındaki ve sonundaki kod değeri aynı değildir, değişkendir. İletişim anında kullanılan yöntemlerdir. Hoca İstanbul'dan Ankara'ya hızın çok önemli olduğu bir veri gönderimi örneğini verdi: "a" önce bir kodla, ilerleyen zamanlarda farklı kodlarla gönderilebilir; karşı taraf (Ankara) her değişimi anında yakalayarak açar.

Bu üç türün nasıl çalıştığı Huffman kodları anlatılırken ayrıntılı gösterilecek.

5.5 Kayıplı ve Kayıpsız Sıkıştırma

Ekrandaki gösterim (ekrandan, 01:40:00):

- **Lossy (kayıplı):** $X \xrightarrow{c} Y \xrightarrow{dc} X'$, $X \neq X'$. Elimizdeki X verisi sıkıştırılıp Y olur; açıldığında elde edilen X', X'ten farklıdır. Bir kayıp söz konusudur.
- **Lossless (kayıpsız):** $X \xrightarrow{c} Y \xrightarrow{dc} X$. Açıldığında birebir aynı veri elde edilir.

Hangisinin kullanılacağı veriye bağlıdır. Metin dosyaları genelde kayıpsız sıkıştırılır. Hocanın örneği: bir dokümanda hesaba "10.000 dolar yatırıldı" yazıyor olsun. Kayıplı sıkıştırma sıfırların bazılarını götürür ve 10.000 dolar 100 dolar olursa sonuç felakettir. Böyle dokümanlarda kayıplı algoritma kullanılamaz.

Kayıplı algoritmalar ses ve görüntüde kullanılır:

- **Ses:** Kulağın algılayamadığı frekans değerleri vardır; o frekansların üstündeki sesleri tutmanın anlamı yoktur. Hoca MPEG'i örnek verdi: sıkıştırılmış dosyada kulağın duyamadığı frekanslar bulunmaz.
- **Görüntü:** True color bir resimde her biri 8 bit olan 3 renk kanalı vardır; $3 \cdot 8 = 24$ bit, yani $2^{24} = 16.777.216$ farklı renk. Hiçbir insan gözü bu kadar rengi ayırt edemez. Görüntü sıkıştırma gözün ayırt edemediği renkleri resimden çıkarıp atmaya yarar.

  Ek bilgi (derste anlatılmadı): MPEG bir standartlar ailesinin adıdır. Yaygın ses biçimi MP3, MPEG-1 Audio Layer III'tür ve büyük olasılıkla hocanın ses örneğinde kastettiği bu ailedir.

Metinde de istisnalar vardır. Bir metindeki çok sayıdaki boşluk tek boşluğa indirilebilir; metnin içeriği değişmez ve kişi bunu kabul edebilir. Biçimli (formatlı) bir metindeki büyük harf, kalın, italik, renk bilgileri atılabilir; içerik bozulmaz. Program kodlarındaki satır arası boşluklar silinse program yine çalışır. Bunlar kayıplı sıkıştırmadır ama hepsi konuya özeldir; hoca bunlara "ad hoc" dedi: kişiye, ortama ve konuya göre değişir.

5.6 Simetrik ve Asimetrik Sıkıştırma

Simetrik sıkıştırmada kompresör ve dekompresör aynı algoritmayı kullanır: sıkıştırırken kullanılan algoritma açarken ters yönde uygulanır, işlemler ters sırayla yapılır. Algoritmalar arasında fark yoktur.

Asimetrik sıkıştırmada kompresör ve dekompresör aynı değildir; algoritmalar birbirinin tersi değildir. Bazen kompresör karmaşık ve uzun süren bir algoritmadır, dekompresör ise basit ve kısa sürede kod çözer. Bazen de tersidir: kompresör hızlı ve basittir, dekompresör uzun süren karmaşık bir algoritmadır.

Hoca iki yön için örnek istedi:

- **Sıkıştırma uzun sürebilir, açma kısa sürmeli:** Arşiv uygulamaları. Bir rapor bir kez sıkıştırılıp arşive konur; bu uzun sürebilir. Ama yüzlerce kişi gün içinde raporu indirdiği için açma tarafı hızlı olmalıdır.
- **Sıkıştırma kısa sürmeli, açma uzun sürebilir:** Öğrencilerden cevap gelmeyince hoca kendisi cevapladı: bankalarda her günün sonunda sistem yedeğe (backup) alınır. Yedekleme her gün yapıldığı için hızlı olmalıdır. Sistemde sorun ise ya hiç çıkmaz ya da yılda bir iki kez çıkar; geri yükleme nadir yapıldığı için açma kısmı daha uzun sürebilir. Hoca "uzun" kelimesinin günler ya da saatler anlamına gelmediğini, örneğin 5 dakikada yapılan işin tersinin 15 dakika sürmesi gibi düşünülmesi gerektiğini söyledi.

5.7 Ara Özet

Hoca kavramları şöyle toparladı: kompresör ve dekompresör ters yönde çalışan iki işlemdir; codec ikisini birden içine alır; ham ve işlenmiş veri ayrılır. Adaptif olmayan algoritmalar belli bir uygulamaya göre yazılmıştır ve parametreleri değişmez; adaptif algoritmalar her türlü veri için çalışır ve sıkıştırma esnasında değişkenleri ve parametreleri değişebilir. Yarı adaptif algoritmalar iki aşamalıdır: önce veri okunup istatistiği çıkarılır, sonra kodlanır. Simetrikte kompresör ve dekompresör birbirinin tersidir; asimetrikte farklıdır ve uygulamaya göre uygun olanı seçilir.

6. Algoritma Seçimi, Akış ve Blok Modu, Fiziksel ve Mantıksal Sıkıştırma

6.1 Neden Bu Kadar Çok Algoritma Var?

Hoca kendi sorusunu sordu: amaç sıkıştırmaksa neden yüzlerce algoritma var; en iyi sıkıştıranı alıp her yerde kullanmak yetmez mi? Cevap: Öyle bir dünya yoktur. Bazı algoritmalar metinde, bazıları görüntüde çok iyi çalışır. Bazıları çok tekrar eden desen varsa daha başarılıdır. Bazıları büyük veride iyi sonuç vermezken bazıları verir.

Seçimde harcanacak zaman da belirleyicidir. Bir işin hızlı yapılması gerekiyorsa, sıkıştırırken ya da açarken, hızlı bir algoritma seçilmelidir. Hız önemli değil de yalnız fiziksel alanın küçülmesi önemliyse sıkıştırma performansı daha iyi olan algoritma seçilmelidir. Bu kararı veren kullanıcıdır. İlk başta rahat karar verilemeyebilir; örneğin verileri hep resim ve fotoğraf olan bir sanatçı, zamanla hangi algoritmanın kendi verisinde daha başarılı olduğunu deneyerek bulur. Bir başka yol da kendi alanındakilerin ne kullandığına bakmaktır, çünkü o denemeyi başkası daha önce yapmıştır.

6.2 Akış Modu ve Blok Modu

Sıkıştırma algoritmaları ya akış modunda (stream mode) ya da blok modunda (block mode) çalışır (ekrandan, 02:00:00):

- **Stream mode:** Veri bit bit, bayt bayt ya da kelime kelime okunur ve sürekli aynı işlem uygulanır; bir değişkenlik yoktur.
- **Block mode:** Veri bloklar hâlinde okunur, örneğin 512 baytlık ya da 1024 baytlık bloklar. Her bloğa ayrı bir sıkıştırma uygulanabilir. Açma için her bloğun başında "bu blok şu algoritmayla kodlandı" ya da "açmak için parametre değerleri şunlardır" gibi bilgiler yazılabilir.

Hoca, Burrows-Wheeler algoritmasının aslında bir dönüşüm (transform) yöntemi olduğunu ve blok modunda çalıştığını söyledi: veriyi bir kerede 512 ya da 1024 olarak çeker ve üzerinde işlem yapar.

6.3 Fiziksel ve Mantıksal Sıkıştırma

Hoca slayta sonradan eklediği bir ayrımı da anlattı: sıkıştırma yöntemleri fiziksel ya da mantıksal (logical) olarak çalışır.

- **Fiziksel sıkıştırma:** Verinin içeriğinin ne olduğu önemli değildir; her şey 1 ve 0'dır. Sıkıştırılmış veriye bakıldığında hiçbir şey anlaşılmaz; açılmadığı sürece anlamaya imkân yoktur.

- **Mantıksal sıkıştırma:** Verinin içinde bir desen aranır ve o desenin yerine başka bir kod konur. Sıkıştırılmış dosyaya bakıldığında kod görülünce hangi desenin yerine geldiği anlaşılır. Örnek: bir programlama kodunda bütün `printf`'ler yerine bir kod, bütün `read`'ler yerine başka bir kod konur. Sıkıştırılmış dosyada o kodu görünce orada `printf` olduğu bilinir. Bunlar da ad hoc uygulamalardır.

7. Sıkıştırma Oranı ve Performansı

7.1 Sıkıştırma Oranı

Sıkıştırmanın başarısını ölçmek için sıkıştırma oranı (compression ratio) kullanılır (ekrandan, 02:03:30):

$$\text{sıkıştırma oranı} = \frac{\text{çıktı akışının boyutu}}{\text{girdi akışının boyutu}}$$

Hocanın örneği:

- **Verilen:** Girdi dosyası 100 birim, sıkıştırılmış dosya 70 birim.
- **Oran:** $70/100 = 0{,}7$.
- **Performans:** Yok olan bölge 30 birimdir; $(1 - 0{,}7) \cdot 100 = \%30$ sıkıştırılmıştır.
- **İkinci durum:** Dosya 30 birime inseydi oran $30/100 = 0{,}3$ olurdu ve $(1-0{,}3)\cdot 100 = \%70$ sıkıştırılmış olurdu.

  Hocanın vurgusu: Sıkıştırma oranını (compression ratio) sıkıştırma performansıyla karıştırmayın. Oran $0{,}7$ ise performans $\%30$'dur.

Sıkıştırma oranı kesinlikle 1'den küçük çıkmalıdır. 1'den büyük çıkarsa, örneğin $1{,}5$, çıktı akışı girdi akışından büyük olmuştur; veri sıkıştırılacağına genişletilmiştir. Yanlış algoritma seçmek bu sonuca yol açabilir.

Bu oran metin sıkıştırmada kullanılır. Hoca görüntüde başka bir performans ölçütü kullanıldığını belirtti (Bölüm 9).

7.2 Bits per Character ve Bits per Pixel

Metinde kullanılan birim **bpc** (bits per character) yani karakter başına bittir: algoritmanın bir karakteri ortalama kaç bitle kodladığı. Bir karakter 1, 2, 3 ya da 15 bitle kodlanabilir; önemli olan ortalamadır. Hoca bunun aslında entropinin birimi olduğunu söyledi: entropi, bir sıkıştırma algoritması kullanıldığında bir sembol için ortalama kaç bit harcandığını söyleyecektir.

Görüntüde birim **bpp** (bits per pixel) yani piksel başına bittir: bir pikseli kodlamak için kaç bit gerektiği.

8. Derlem (Corpus)

Korpusun Türkçesi derlemdir; Arapçadan gelen "külliyat" da kullanılır. Derlem, üzerinde çalışılan veri türünden oluşan bir topluluktur: veri metin ise dokümanlardan, görüntü ise görüntülerden, ses ise seslerden oluşan bir topluluk.

Bilinen metin derlemleri **Calgary** ve **Canterbury** derlemleridir (ekrandan, 02:06:40). Calgary derleminde obje (obj) dosyaları, metin dosyaları gibi çok farklı içerikte dosyalar vardır. Canterbury derlemi bunu geliştirmiştir ve daha geniştir; Calgary hazırlandığı dönemde bulunmayan PowerPoint, HTML, Lisp gibi dosya türlerini de içerir.

> **Ders sonrası not:** Hoca iki derlemin de Kanada'daki üniversitelerde hazırlandığını söyledi. Calgary derlemi büyük olasılıkla Kanada'daki University of Calgary'de, Canterbury derlemi ise büyük olasılıkla Yeni Zelanda'daki University of Canterbury'de hazırlanmıştır.

Derlemin neden gerekli olduğunu hoca şöyle açıkladı: Ortaya atılan bir yöntemin performansı herkesin kullandığı bir derlem üzerinde ölçülmelidir. Biri kendi hazırladığı, algoritmasının çok iyi çalışacağı verilerle "algoritmam %90 başarılı" derse kimse inanmaz, çünkü düzmece veri

hazırlanmadığı bilinemez. Bu yüzden standart derlemler hazırlanır, herkes algoritmasını bu derlemler üzerinde test eder ve sonuçları birbiriyle karşılaştırır.

Görüntü üzerinde çalışılıyorsa da böyle bir veri seti vardır. Hoca adını "UC" diye andı; içinde siyah-beyaz, gri seviyeli, renk tablolu (256 renk) ve true color resimler, çok karmaşık ve farklı farklı resimler bulunduğunu söyledi. Sesle çalışılıyorsa da temel alınacak (baseline) ses veri setleri gerekir.

Soru: Derlem dediğimiz şey, veri sıkıştırma için kullanılan veri setlerinin bulunduğu bir kütüphane mi? **Cevap:** Evet. Hoca ayrıca derlemin yalnız sıkıştırmaya özgü olmadığını ekledi. Örneğin doğal dil işlemede yazar tanıma yapan biri 100 yazarın 50'şer dokümanını toplayıp bir derlem oluşturabilir ve başkalarıyla paylaşabilir; algoritmasını onunkiyle yarıştırmak isteyen bu veriyi kullanır. Doküman sınıflandırmada Reuters derlemi ya da 20 Newsgroups derlemi kullanılır. Kısacası derlem bir deney setidir: herkes algoritmasını aynı girdilerle çalıştırır ki başarılar kıyaslanabilsin.

9. Görüntü Sıkıştırmada Performans Ölçümü

9.1 Görüntüde Sıkıştırma Neyi Değiştirir?

Görüntü satırlardan (row) ve sütunlardan (column) oluşur; $M \times N$ boyutlu bir görüntüye I diyelim. Görüntü sıkıştırılıp açıldığında elde edilen görüntüye $\hat{I}$ ("I şapka") denir (ekrandan, 02:20:00).

> **Hocanın vurgusu:** Bir görüntüyü sıkıştırmak boyutlarını değiştirmek demek değildir. 100×100'lük bir resim sıkıştırıldığında yine 100×100 olur.

Değişen, kullanılan renk sayısıdır. Örneğin resimde 256 farklı renk kullanılıyorsa sıkıştırılmış hâlde 64 renk kullanılabilir. Hoca bunu yan yana iki kırmızı renk örneğiyle açıkladı: göz bu iki çok yakın kırmızıyı ayırt edemiyorsa ikisi de aynı kırmızı olarak alınır. Kullanılan sembol sayısı azaldıkça daha az kod kullanılır ve veri daha kolay sıkıştırılır.

9.2 Neden Yalnız Oran Yetmez?

Metinde çıktı boyutu girdi boyutuna bölünüyordu. Görüntüde bu tek başına yetmez, çünkü görüntünün bozulup bozulmadığı da ölçülmelidir. Kayıplı bir algoritma resmi öyle sıkıştırabilir ki açıldığında orijinalle ilgisi kalmaz; ekrana bir şey gelir ama ne olduğu anlaşılmaz. Bu yüzden görüntüde orijinal ile açılan görüntü arasındaki hata ölçülür.

9.3 Hata, RMS Hata ve SNR

Her piksel için hata, açılan görüntüdeki değer ile orijinal değer arasındaki farktır (ekrandan, 02:20:00):

$$\text{error}(r, c) = \hat{I}(r, c) - I(r, c)$$

Toplam hata bütün satırlar ($r = 0, \dots, N-1$) ve sütunlar ($c = 0, \dots, M-1$) üzerinden hesaplanır; satır ve sütunlar sıfırdan başladığı için üst sınırlar $N-1$ ve $M-1$'dir. Ortalama karesel hatanın karekökü (root mean square error) şöyledir:

$$e_{RMS} = \sqrt{\frac{1}{N \cdot M} \sum_{r=0}^{N-1} \sum_{c=0}^{M-1} \left[\hat{I}(r, c) - I(r, c)\right]^2}$$

Hoca işlemi basitleştirmek için görüntünün $N \times N$ kabul edilebileceğini de söyledi. e_{RMS} ne kadar küçükse sıkıştırılmış resim açıldığında orijinale o kadar yakındır.

e_{RMS}'in tersi davranan ölçüt sinyal-gürültü oranıdır (Signal-to-Noise Ratio, SNR) (ekrandan, 02:22:55):

$$SNR_{RMS} = \sqrt{\frac{\sum_r \sum_c \hat{I}(r,c)^2}{\sum_r \sum_c \left[\hat{I}(r,c) - I(r,c)\right]^2}}$$

Pay, sıkıştırılmış (açılmış) resimdeki her pikselin karesinin toplamıdır; payda toplam hatadır. İkisi ters orantılıdır: SNR küçükse e_{RMS} büyük, e_{RMS} küçükse SNR büyük çıkar. Bazı çalışmalar e_{RMS}, bazıları SNR verir; ters orantılı oldukları bilinmelidir.

SNR'nin bir de tepe (peak) biçimi vardır, PSNR; değeri logaritmik olarak alınır (ekrandan, 02:22:55):

$$PSNR = 10 \log_{10} \frac{(L-1)^2}{\frac{1}{NM} \sum_r \sum_c \left[\hat{I}(r,c) - I(r,c)\right]^2}$$

Burada L, resimde kullanılan renk sayısıdır.

> **Ders sonrası not:** Ekranda "total error" farkların kare alınmadan toplamı olarak yazıldı ($\sum \sum [\hat{I}(r,c) - I(r,c)]$) ve e_{RMS} ile SNR_{RMS} bu toplamla kuruldu. "Root mean square" adı gereği farkların karesi alınır; kare alınmazsa pozitif ve negatif hatalar birbirini götürebilir. Bu notta formüller büyük olasılıkla doğru olan kareli biçimde yazıldı. Hoca da formülleri daha düzgün yazıp paylaşacağını söyledi.
>
> **Ek bilgi (derste anlatılmadı):** 8 bitlik gri seviyeli görüntülerde $L = 256$ olduğundan PSNR formülündeki pay 255^2 olur. PSNR desibel (dB) cinsinden verilir; değer büyüdükçe açılan görüntü orijinale daha yakındır.

Özetle görüntüde performans e_{RMS} ya da SNR değerlerine bakılarak yorumlanır. e_{RMS} ne kadar küçükse o kadar iyi bir sıkıştırma yöntemi seçilmiştir; açılan resim orijinale çok yakındır. e_{RMS} büyükse açılan resim orijinalden çok uzaktır, belki bulanıktır: sıkıştırma uğruna resim bozulmuştur.

Örnek Sorular

Soru 1 (ekrandan, 02:00:00; derste soruldu 01:53:00): İyi bir sıkıştırma yönteminin kullanılmasının önemli olduğu fakat kodlama ve kod çözme hızının önemli olmadığı bir duruma örnek veriniz.

> **Çözüm (derste verildi):** Bir öğrencinin cevabından yola çıkan hoca şu örnekleri verdi: bilgisayara indirilen bir programı sıkıştırıp saklamak; sıkıştırma ne kadar iyi olursa diskte o kadar az yer kaplar, sıkıştırma ve açma süresi kullanıcı için önemli değildir. Ya da yazılan bir uygulamayı Drive'da paylaşmak; önemli olan uygulamanın az yer kaplamasıdır, bir kez konur ve indirenin ne kadar beklediği önemli değildir.

Soru 2 (derste sözlü, 01:44:40): Sıkıştırma işleminin uzun sürmesinin önemli olmadığı, fakat açma işleminin çok kısa sürmesi gereken bir uygulama alanı nedir?

> **Çözüm (derste verildi):** Arşiv uygulamaları. Dosya arşive bir kez konur, bu uzun sürebilir; ama dosya gün içinde pek çok kişi tarafından indirildiği için açma hızlı olmalıdır.

Soru 3 (derste sözlü, 01:46:30): Sıkıştırma işleminin çok kısa sürmesi gereken, açma işleminin ise daha uzun sürebileceği bir günlük hayat örneği veriniz.

> **Çözüm (derste verildi):** Bankalarda gün sonu yedekleme (backup). Yedek her gün alındığı için sıkıştırma hızlı olmalıdır; geri yükleme yılda bir iki kez gerektiği için açma daha uzun sürebilir.

Soru 4 (derste sözlü, 02:04:00): Metinde sıkıştırma performansı çıktı/girdi oranıyla ölçülüyor. Görüntüde performans nasıl ölçülür, formül nasıl olur?

Çözüm (derste verildi): Görüntüde yalnız oran yetmez, çünkü açılan görüntünün orijinale ne kadar yakın olduğu da ölçülmelidir. Bunun için e_{RMS} ve SNR (ya da PSNR) kullanılır; formüller Bölüm 9.3'tedir. e_{RMS} küçük, SNR büyük olmalıdır.

Dersin Toparlaması

Bu ilk derste veri sıkıştırmanın ne olduğu, neden gerektiği ve dersin geri kalanında sürekli kullanılacak kavramlar işlendi. Sıkıştırma, girdi akışını kompresörle küçültüp çıktı akışına yazmak ve dekompresörle geri açabilmektir; amaç alan ve hız kazancıdır. Sabit uzunluklu kodlar (ASCII) sembol sıklıklarını kullanmazken değişken uzunluklu kodlar sık sembollere kısa kod vererek sıkıştırma sağlar; 30 "a" ve 2 "j" örneğinde 256 bit 108 bite indi. Artıklık, veride hâlâ sıkıştırılabilecek kısım olup olmadığını gösterir ve art arda sıkıştırmanın durma noktasını belirler. Bunlara adaptif ve adaptif olmayan, statik, yarı dinamik ve dinamik, kayıplı ve kayıpsız, simetrik ve asimetrik sıkıştırma, akış ve blok modu, fiziksel ve mantıksal sıkıştırma ayrımları eklendi. Son olarak başarının ölçümü ele alındı: metinde sıkıştırma oranı ve bpc, görüntüde bpp, e_{RMS}, SNR ve PSNR; karşılaştırmaların ise Calgary ve Canterbury gibi standart derlemler üzerinde yapılması gerektiği vurgulandı.

Artıklığın biçimsel tanımı ve bpc biriminin asıl anlamı, ilerideki bilgi teorisi bölümünde entropi ile birlikte verilecek. Statik, yarı dinamik ve dinamik kodlamanın işleyişi Huffman kodlarında, faks sıkıştırması ise adaptif olmayan sıkıştırmanın örneği olarak ayrıca gösterilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.